

데이터베이스 (DataBase)

전북대학교 컴퓨터인공지능학부

이경수

14

CHAPTER

MongoDB Practice

MongoDB Practice

다음 요구명세서를 기반으로 MongoDB와 RDB를 설계해보세요.

1. 이 시스템은 사용자가 블로그 형식의 게시글을 작성하고, 댓글을 주고받으며, 추천(좋아요) 등의 상호작용을 가능하게 하는 콘텐츠 플랫폼을 목표로 합니다.
2. **사용자:** 사용자는 블로그의 기본 단위 주체로, 고유한 ID와 함께 사용자명, 이메일, 해시된 비밀번호를 갖습니다. 프로필 이미지와 역할(예: 관리자, 일반 사용자 등)을 포함하며, 가입 일시는 기록됩니다.
3. **게시물 (POST):** 게시물은 사용자가 작성한 콘텐츠로, 제목과 본문, 썸네일 이미지, 작성자 정보 등을 포함합니다. 게시물에는 카테고리, 다수의 태그, 첨부 이미지 목록, 추천 수, 댓글 등의 정보가 포함될 수 있습니다. 게시물은 공개/비공개/임시 상태를 가질 수 있습니다.
4. **댓글:** 댓글은 게시물에 달린 사용자 피드백으로, 본문 내용과 작성자 정보, 작성일 등을 포함합니다. 대댓글 기능을 위해 부모 댓글 ID를 선택적으로 포함합니다.
5. **추천:** 추천은 사용자와 게시물 간의 좋아요 관계를 나타냅니다. 한 사용자가 한 게시물을 한 번만 추천할 수 있도록 제한합니다.
6. **카테고리:** 카테고리는 게시물 분류를 위한 기준입니다. 단순한 이름과 설명 필드로 구성되며, 대부분의 경우 사전 정의된 카테고리만 사용
7. **태그:** 태그는 게시물의 검색성과 분류를 위한 키워드이며, 여러 개가 게시물에 연결됩니다. 태그를 별도 문서로 관리하거나 단순 문자열 배열로 게시물 내부에 포함시킬 수 있습니다.

MongoDB Practice

사용자: 사용자는 블로그의 기본 단위 주체로, 고유한 ID와 함께 사용자명, 이메일, 해시된 비밀번호를 갖습니다. 프로필 이미지와 역할(예: 관리자, 일반 사용자 등)을 포함하며, 가입 일시는 기록됩니다.

```
const User = require('./models/User');

async function createUser() {
  const user = new User({
    username: 'john_doe',
    email: 'john@example.com',
    password_hash: 'hashed_password',
    role: 'user'
  });

  const savedUser = await user.save();
  console.log('User saved:', savedUser);
}
```

MongoDB Practice

게시물 (POST): 게시물은 사용자가 작성한 콘텐츠로, 제목과 본문, 썸네일 이미지, 작성자 정보 등을 포함합니다. 게시물에는 카테고리, 다수의 태그, 첨부 이미지 목록, 추천 수, 댓글 등의 정보가 포함될 수 있습니다. 게시물은 공개/비공개/임시 상태를 가질 수 있습니다.

```
const Post = require('./models/Post');

async function createPost(authorId) {
  const post = new Post({
    author_id: authorId,
    title: '첫 번째 게시물입니다',
    content: '이것은 MongoDB를 사용하는 예시입니다.',
    category: '기술',
    tags: ['mongodb', 'nodejs'],
    is_published: true
  });

  const savedPost = await post.save();
  console.log('Post saved:', savedPost);
}
```

MongoDB Practice

댓글: 댓글은 게시물에 달린 사용자 피드백으로, 본문 내용과 작성자 정보, 작성일 등을 포함합니다. 대댓글 기능을 위해 부모 댓글 ID를 선택적으로 포함합니다.

```
const Comment = require('./models/Comment');

async function createComment(postId, authorId) {
  const comment = new Comment({
    post_id: postId,
    author_id: authorId,
    content: '정말 유익한 글이네요!'
  });

  const savedComment = await comment.save();
  console.log('Comment saved:', savedComment);
}
```

MongoDB

추천: 추천은 사용자와 게시물 간의 좋아요 관계를 나타냅니다. 한 사용자가 한 게시물을 한 번만 추천할 수 있도록 제한합니다.

```
const Post = require('./models/Post');

async function likePost(postId, userId) {
  const post = await Post.findById(postId);
  if (!post) {
    console.log('게시물을 찾을 수 없습니다.');
```

// 이미 좋아요를 눌렀는지 확인

```
    return;
  }

  if (post.likes.includes(userId)) {
    console.log('이미 이 게시물에 좋아요를 눌렀습니다.');
```

// 좋아요 추가

```
    return;
  }

  post.likes.push(userId);
  await post.save();
  console.log('좋아요 추가 완료:', post.likes.length);
}
```

MongoDB Practice

추천: 추천은 사용자와 게시물 간의 좋아요 관계를 나타냅니다. 한 사용자가 한 게시물을 한 번만 추천할 수 있도록 제한합니다.

```
const LikeSchema = new mongoose.Schema({
  user_id: { type: mongoose.Schema.Types.ObjectId, ref: 'User' },
  post_id: { type: mongoose.Schema.Types.ObjectId, ref: 'Post' },
  liked_at: { type: Date, default: Date.now }
});

// 좋아요 수 세기
const likeCount = await Like.countDocuments({ post_id: somePostId });
```


MongoDB Practice

카테고리: 카테고리는
이름과 설명 필드로
카테고리만 사용

```
const Post = require('./models/Post');
const Category = require('./models/Category');

async function createPostWithCategory(userId) {
  // 1. 사용할 카테고리 ID 찾기 또는 생성
  let category = await Category.findOne({ name: '프로그래밍' });
  if (!category) {
    category = await new Category({ name: '프로그래밍', description: '개발 관련 글' }).save();
  }

  // 2. 게시물 생성
  const post = new Post({
    author_id: userId,
    title: 'Node.js로 MongoDB 연동하기',
    content: 'Mongoose를 통해 쉽게 사용할 수 있어요.',
    category: category._id,
    tags: ['nodejs', 'mongoose'],
    is_published: true
  });

  const saved = await post.save();
  console.log('게시글 생성 완료:', saved);
}
```

MongoDB Practice

카테고리: 카테고리는 게시물 분류를 위한 기준입니다. 단순한 이름과 설명 필드로 구성되며, 대부분의 경우 사전 정의된 카테고리만 사용

```
const Post = require('./models/Post');

async function getAllPostsWithCategory() {
  const posts = await Post.find()
    .populate('author_id', 'username')
    .populate('category', 'name') // 카테고리 이름만 포함
    .sort({ created_at: -1 });

  console.log(posts);
}
```

MongoDB Practice

카테고리: 카테고리는 게시물 분류를 위한 기준입니다. 단순한 이름과 설명 필드로 구성되며, 대부분의 경우 사전 정의된 카테고리만 사용

```
async function getPostsByCategory(categoryName) {  
  const category = await Category.findOne({ name: categoryName });  
  if (!category) return [];  
  
  const posts = await Post.find({ category: category._id }).populate('author_id', 'username');  
  return posts;  
}
```

MongoDB Practice

예제 케이스 1: 게시물에 댓글을 중첩 문서로 저장할 때

- 댓글 수가 적고, 댓글을 자주 개별적으로 조회하거나 수정하지 않는 경우
 - 댓글을 Post 문서 안의 배열(comments)로 저장하는 것이 효율적입니다.
 - MongoDB는 배열 안의 객체를 빠르게 조회/업데이트 할 수 있고, 하나의 게시글을 조회할 때 댓글까지 같이 불러올 수 있어 조인 비용이 없음.

```
{
  "_id": ObjectId("..."),
  "title": "MongoDB 내장 댓글 예제",
  "content": "...",
  "comments": [
    {
      "user_id": ObjectId("..."),
      "content": "좋은 글 감사합니다!",
      "created_at": ISODate("2025-06-04T00:00:00Z")
    },
    ...
  ]
}
```

MongoDB Practice

예제 케이스 2: 게시물 추천을 사용자 ID 배열로 저장

- "누가 추천했는지"가 중요하지만, 추천 수만 빠르게 세거나 중복 추천 방지하는 것이 주 목적일 때,
- likes: [user_id] 배열로 저장하는 것이 관계형 DB보다 훨씬 효율적!

```
{
  "_id": ObjectId("..."),
  "title": "좋아요 예제",
  "likes": [ObjectId("user1"), ObjectId("user2")]
}
```

```
async function likePost(postId, userId) {
  await Post.updateOne(
    { _id: postId, likes: { $ne: userId } }, // 중복 방지
    { $push: { likes: userId } }
  );
}
```

```
const post = await Post.findById(postId);
console.log('추천 수:', post.likes.length);
```

MongoDB Practice

예제 케이스 3: User → Post → Comment → Like까지 연결하려는 경우

- MongoDB는 관계형 DB처럼 자유로운 JOIN을 지원하지 않음!
- \$lookup으로 컬렉션 간 조인을 하긴 하지만, 성능이 떨어지거나 쿼리가 복잡해짐.
- 이와 같이 여러 단계로 JOIN이 쌓이면 MongoDB는 속도와 가독성 모두 저하됨.

```
// 댓글을 조회하면서, 댓글 작성자, 댓글이 속한 게시글, 게시글의 작성자까지 가져오기
const comments = await Comment.aggregate([
  {
    $lookup: {
      from: 'posts',
      localField: 'post_id',
      foreignField: '_id',
      as: 'post'
    }
  },
  { $unwind: '$post' },
  {
    $lookup: {
      from: 'users',
      localField: 'post.author_id',
      foreignField: '_id',
      as: 'post_author'
    }
  }
]
// ...계속 늘어나는 JOIN
]);
```

MongoDB Practice

도큐먼트 중심 모델링 (Document-Oriented Modeling)

- MongoDB는 관계형 데이터베이스 (RDBMS)와 달리 테이블이 아닌 문서(document) 단위로 데이터를 저장합니다. 따라서 가능하면 관련 정보를 하나의 문서 안에 중첩(subdocument) 형태로 묶어 관리
 - 예: 게시물(*Post*) 내에 이미지 목록을 배열로 포함하거나, 추천한 사용자 ID 목록(*likes*)을 포함.
 - 장점: 조회 속도가 빠르고, 게시물 하나만 읽어도 관련 정보를 대부분 확보할 수 있음 (read-optimized).

MongoDB Practice

중첩(nested) vs 참조(referenced)

- MongoDB에서는 중첩 구조와 참조 구조를 상황에 따라 선택
- 중첩 문서 사용
 - 관련 데이터가 항상 함께 조회되는 경우: 예를 들어 이미지 목록이나 태그 등.
 - 크기가 작고 변화가 많지 않은 구조에 적합.
- 참조 문서 사용
 - 재사용되거나 수정/확장될 가능성이 높은 경우: 사용자, 댓글, 카테고리 등.
 - 독립적으로 관리되거나 개별 조회가 자주 필요한 경우.

MongoDB Practice

중첩(nested) vs 참조(referenced)

- MongoDB에서는 중첩 구조와 참조 구조를 상황에 따라 선택

결정 기준 요약:

- 중첩 문서

- 관련 데이터가 많을 때
- 크기가 작을 때

상황	중첩 구조 사용	참조 구조 사용
함께 자주 조회됨	✓	✗
문서 크기 커지거나 자주 수정됨	✗	✓

- 참조 문서

- 재사용
- 독립적으로 관리되거나 개별 조회가 자주 필요한 경우.

독립적 관리 또는 공유 필요	✗	✓
-----------------	---	---

MongoDB Practice

읽기(read) 중심 최적화

- MongoDB는 읽기 성능을 높이기 위해 데이터를 중복을 포함(denormalization)하는 것을 권장
 - 예: 게시물에 작성자 ID뿐 아니라, 작성자 이름이나 프로필 이미지 URL도 함께 저장 가능 (캐싱 형태).
 - 이유: 사용자 정보가 자주 조회되지만 자주 변경되지는 않기 때문.
 - 주의: 다만 사용자 정보가 변경되면 여러 문서를 함께 갱신해야 하므로 업데이트 로직이 복잡해질 수 있음.

MongoDB Practice

스키마 유연성 (Flexible Schema)

- MongoDB는 스키마가 엄격하지 않기 때문에, 새로운 필드 추가나 기능 확장이 용이
 - 예: 향후 "북마크", "신고", "공유 이력", "태그 색상" 같은 기능이 생겨도 기존 문서에 쉽게 필드를 추가할 수 있음.
 - 단점: 일관성 유지를 위해 Mongoose 같은 ODM(Object Document Mapper) 또는 JSON Schema를 활용한 구조 검증 권장.

MongoDB Practice

정규화 vs 비정규화 (Normalization vs Denormalization)

- RDBMS에서는 정규화를 통해 데이터 중복을 줄이지만, MongoDB에서는 읽기 효율을 위해 비정규화(denormalization) 전략을 더 많이 사용
- 예:
 - 정규화: Post → PostTag → Tag 식으로 참조
 - 비정규화: Post.tags = ["MongoDB", "NoSQL"] 식으로 배열 저장

MongoDB Practice

확장성과 성능 고려

- 조회수, 추천수, 댓글수 등은 집계용 필드로 문서에 포함하여 빠르게 보여줄 수 있도록 설계
- 이러한 수치는 실제 데이터(예: likes 배열)와 동기화가 필요하므로, 증가/감소시 carefully handled.

Thanks!

질문은 여기에...

ksl@jbnu.ac.kr

